

VitaZen

FASE 5.2

Optimizacion Segura de Rendimiento

*Indices de base de datos, paginacion de consultas Prisma,
agregacion en DB para analytics, limites de seguridad*

16 de julio de 2026

1. Resumen Ejecutivo

Este informe documenta la auditoria forense de rendimiento y las optimizaciones implementadas en la FASE 5.2 de VitaZen. El objetivo principal fue identificar y corregir consultas de base de datos sin limitar, operaciones de agregacion ineficientes en memoria, y riesgos de seguridad en parametros de entrada, todo ello bajo la restriccion estricta de no modificar la arquitectura existente ni introducir cambios visuales. Se priorizaron optimizaciones de bajo riesgo y alto beneficio que pudieran implementarse sin riesgo de regresion.

La auditoria cubrio 60 archivos de ruta API y 16 archivos de libreria, identificando 11 consultas findMany sin paginar, 15 casos de sobre-fetching de campos, y un problema critico de uso de memoria en el endpoint de analytics. Como resultado, se implementaron 10 optimizaciones concretas y se corrigieron 3 errores de sintaxis en el schema Prisma que impedian la validacion. Todas las optimizaciones mantienen la compatibilidad total con el frontend existente y no introdujeron ningun error nuevo de TypeScript (los 32 errores preexistentes permanecen sin cambios).

Archivos auditados	76 (60 rutas API + 16 lib)
Optimizaciones implementadas	10 cambios en 9 archivos
Errores de schema corregidos	3 (modelos sin cierre y @@index fuera de modelo)
Nuevos errores TypeScript	0 (32 preexistentes sin cambios)
Riesgo de regresion	Minimo (cambios aditivos: take, clamp, groupBy)

2. Metodologia de Auditoria

La auditoria se ejecuto en tres fases secuenciales. En primer lugar, se leyó y analizo el archivo prisma/schema.prisma completo para catalogar todos los indices existentes y verificar su cobertura frente a los patrones de consulta observados en el código. En segundo lugar, se leyeron los 60 archivos de ruta API y los 16 archivos de libreria que contienen acceso a base de datos, evaluando cada consulta findMany para determinar si tiene paginacion (take/skip/cursor), si usa select para limitar campos, si tiene manejo de errores try/catch, y si presenta patrones N+1. En tercer lugar, se evaluaron los puntos criticos de memoria, especialmente el endpoint de analytics que carga todos los eventos del periodo en memoria para agregarlos en JavaScript.

Cada hallazgo se clasifico en siete categorias: (A) findMany sin paginar, (B) falta de try/catch, (C) sobre-fetching de campos, (D) consultas N+1, (E) problemas de memoria en analytics, (F) falta de transacciones, y (G) ya optimizado. Solo las optimizaciones de categoria A, C y E se implementaron, ya que las categorias B, D y F presentaban riesgo insuficiente o requerian refactorizacion arquitectonica que estaba expresamente prohibida por la especificacion de la FASE 5.2.

3. Estado de Indices de Base de Datos

El schema de Prisma ya contiene un conjunto robusto de indices, muchos de ellos anadidos en FASE anteriores (GLOBAL-18, PERF-5.2 previo). Se identificaron 22 @@index y 8 @@unique que cubren los patrones de consulta mas frecuentes del proyecto. Los indices compuestos clave incluyen: userId+archived en AIThread para la lista de conversaciones, threadId+createdAt en AIMessage para la carga ordenada de mensajes, userId+date en multiples modelos de tracking (DailyCheckin, WellnessLog, FinanceLog) para consultas por rango de fechas, y userId+active en PushToken para la busqueda de dispositivos activos de notificacion.

No se anadieron nuevos indices en esta FASE porque la cobertura existente es completa para los patrones de consulta actuales. Los tres @@index marcados con PERF-5.2 en FASEs anteriores (User.weeklyEmailSummary+emailVerified, User.dailyReminders, AnalyticsEvent.userId+event+createdAt, WidgetSnapshot.expiresAt, WidgetSnapshot.userId+computedAt, EmpireTip.empire+plan, y otros) ya estan desplegados en Neon. La unica intervencion en el schema fue la correccion de 3 errores de sintaxis: dos modelos (Subscription, UserChallenge) carecian de su llave de cierre "}", y el indice @@index([empire, plan]) de EmpireTip estaba posicionado fuera del modelo. Estos errores hacian que prisma validate fallara con 17 errores en cascada.

4. Optimizaciones Implementadas

4.1. Journal GET — Limitacion de resultados (take: 100)

El endpoint GET /api/journal carecia de cualquier limite en la consulta findMany, retornando todas las entradas de diario del usuario ordenadas por createdAt descendente. Cada entrada incluye campos de texto pesados como "content" (potencialmente 4000+ caracteres) y "gratitude". Un usuario activo con cientos de entradas generaba respuestas HTTP de megabytes. La solucion fue anadir take: 100, que limita la lista a las 100 entradas mas recientes. Las entradas individuales se cargan bajo demanda mediante PUT o busqueda por ID, por lo que el frontend no necesita todas las entradas en la carga inicial. Este cambio es seguro porque la vista de lista del frontend solo muestra titulo, estado de animo y fecha para cada entrada.

4.2. Wellness GET — Restriccion del parametro days

El endpoint GET /api/wellness aceptaba un parametro de consulta "days" sin limite superior. El parseo directo parseInt(searchParams.get("days") || "30") permitia valores arbitrariamente grandes como days=999999, lo que generaria una consulta masiva sobre todos los registros de bienestar del usuario desde hace mas de 2700 anos. La ruta de finanzas ya tenia la proteccion correcta con Math.min(Math.max(parsed, 10), 365). Se aplico el mismo patron: Math.min(Math.max(parseInt(...), 1), 365), asegurando un minimo de 1 dia y un maximo de 365. El cambio es compatible con el frontend que ya envia valores tipicos de 7 a 90 dias.

4.3. AI Messages — Cap de seguridad para usuarios PREMIUM

El endpoint `GET /api/ai/threads/[threadId]/messages` tenia dos rutas: los usuarios `FREE` veian los ultimos 50 mensajes (con `take: 50`), pero los usuarios `PREMIUM` no tenian ningun limite, cargando todos los mensajes del hilo con todos los campos. En conversaciones largas de meses, esto podia significar cientos de mensajes con contenido completo. Se anadio `MESSAGES_LIMIT_PREMIUM = 500` como cap de seguridad. Este valor cubre mas de 6 meses de conversacion diaria intensa (2-3 intercambios por dia) y es lo suficientemente alto para no afectar la experiencia de ningun usuario real, mientras previene escenarios degenerativos. Los mensajes se ordenan ascendentemente para mantener la compatibilidad con el chat existente.

4.4. AI Threads — Cap y select para lista de conversaciones

El endpoint `GET /api/ai/threads` tenia un problema dual. Primero, la variable `threadLimit` era undefined para usuarios `PREMIUM` (`isPremium ? undefined : HISTORY_LIMIT_FREE`), lo que significaba que un usuario `PREMIUM` con 100+ hilos activos recibia todos con sus mensajes incluidos. Segundo, el include de mensajes retornaba todos los campos del ultimo mensaje (`role`, `content`, `createdAt`), pero el frontend de la lista de hilos solo usa `"role"` y `"createdAt"` para mostrar el preview. Se cambio `threadLimit` a `isPremium ? MAX_THREADS_PREMIUM (100) : HISTORY_LIMIT_FREE (10)`, alineandolo con el limite de creacion de hilos ya existente. Ademas, se anadio `select: { role: true, createdAt: true }` al include de mensajes, eliminando la transferencia innecesaria del campo `"content"` que puede contener respuestas de IA de miles de caracteres. Se verifico mediante busqueda en el codigo que ningun componente del frontend accede a `messages[0].content` en la vista de lista de hilos.

4.5. Habits GET — Limitacion a 100 habitos

El endpoint `GET /api/habits` retornaba todos los habitos del usuario sin limite. Si bien es inusual que un usuario tenga mas de 50 habitos activos, la ausencia de un `take` creaba un riesgo de crecimiento descontrolado. Se anadio `take: 100` como cap de seguridad, cubriendo todos los casos de uso realistas (un usuario disciplinado gestionaria tipicamente 10-30 habitos). La consulta ya estaba ordenada por `createdAt` descendente, por lo que los habitos mas recientes son los primeros en retornarse. Este cambio no afecta la funcionalidad de completado de habitos (`PATCH`) que opera por ID individual.

4.6. Patterns y Life Memory — Limitacion de habitLog

Dos endpoints de inteligencia, `/api/patterns` y `/api/life-memory`, ejecutaban la misma consulta sin limite: `db.habitLog.findMany({ where: { userId }, select: { name: true, streak: true, lastCompletedAt: true } })`. Estos endpoints usan los datos de habitos para deteccion de patrones y construccion de la linea de vida, respectivamente. En ambos casos, solo se necesitan los habitos recientes y activos para el analisis. Se anadio `take: 100` a ambas consultas. Las demas consultas en estos endpoints (`financeLogs`, `wellnessLogs`, `meditationSessions`, `checkins`, `journalEntries`) ya estaban correctamente acotadas por fechas con `ninetyDaysAgo` mediante `select` para limitar campos.

4.7. Analytics Insights — Agregacion en base de datos (optimizacion critica)

[OPTIMIZACION DE MAYOR IMPACTO]

El endpoint `GET /api/analytics/insights` presentaba el problema de rendimiento mas critico de todo el proyecto. La implementacion original cargaba TODOS los eventos de analytics del periodo (hasta 90 dias) en memoria del servidor usando `findMany`, y luego agregaba en JavaScript usando bucles `for` con Sets anidados. Con 1000

usuarios generando 5 eventos por día durante 90 días, esto significaba 450,000 filas cargadas en memoria, mas la creacion de objetos Set por cada tipo de evento y por cada día para calcular DAU. En un escenario de crecimiento moderado a 10,000 usuarios, la memoria necesaria superaria los 4.5 millones de filas, causando OOM kills y timeout de la funcion serverless.

La solución reemplaza las 4 operaciones en memoria por 5 consultas SQL dirigidas que delegan la agregación a PostgreSQL. Primero, `db.analyticsEvent.groupBy({ by: ["event"] })` reemplaza el bucle de conteo de eventos por tipo. Segundo, `db.analyticsEvent.groupBy({ by: ["event", "userId"] })` calcula usuarios únicos por tipo de evento sin cargar filas individuales. Tercero, una consulta SQL raw con `GROUP BY DATE("createdAt")` y `"userId"` calcula la tendencia DAU directamente en la base de datos. Cuarto, `db.analyticsEvent.groupBy({ by: ["userId"] })` obtiene el total de usuarios únicos del periodo. Quinto, `db.analyticsEvent.count()` reemplaza `events.length` para el total. El resultado neto es una reducción de complejidad de $O(N)$ en memoria a $O(K)$ donde K es el número de tipos de evento distintos (típicamente menos de 20), con toda la carga de procesamiento en PostgreSQL que está optimizado para este tipo de operaciones.

4.8. Achievements — Limitación de `allCheckinDates`

La función `checkAndAwardAchievements` en `src/lib/achievements.ts` ejecutaba una consulta `findMany` sin límite para obtener todas las fechas de check-in del historial completo del usuario. Esta consulta se usa para detectar gaps en el historial (logros de "comeback") y contar meses distintos. Un usuario con 3 años de check-ins diarios generaría 1095 filas. Si bien la consulta ya usaba `select: { date: true }` para evitar campos pesados, la ausencia de un `take` creaba un riesgo teórico de crecimiento ilimitado. Se añadió `take: 1095` (3 años de datos diarios), que cubre todos los escenarios realistas de detección de gaps. Los logros de comeback requieren detectar ausencias de 14+ días, lo cual es perfectamente cubierto con 3 años de historial. Este cambio no afecta ningún logro activo porque el usuario más antiguo de VitaZen tiene menos de 1 año de datos.

4.9. Correcciones del Schema Prisma

Durante la validación post-implementación con `prisma validate`, se descubrieron 3 errores de sintaxis en `prisma/schema.prisma` que causaban 17 errores en cascada. El primer error: el modelo `Subscription` (línea 106) carecía de su llave de cierre `"}"`. El comentario `@@index` contenía un carácter `"}` al final que fue removido en una FASE anterior, pero ese `"}` era también la llave de cierre del modelo. El segundo error: el modelo `UserChallenge` (línea 182) tenía el mismo problema. El tercer error: `@@index([empire, plan])` del modelo `EmpireTip` estaba posicionado después de la llave de cierre del modelo, fuera de su ámbito. Las tres correcciones fueron simples: agregar las llaves de cierre faltantes y mover el `@@index` dentro del modelo. Después de las correcciones, `prisma validate` confirmó "The schema is valid" y `prisma db push` sincronizó correctamente con Neon.

5. Hallazgos Auditados pero No Implementados

Durante la auditoría se identificaron hallazgos adicionales que no se implementaron por cumplir con la regla estricta de la FASE 5.2: "Si durante la implementación detectas que alguna optimización requiere modificar la

arquitectura, refactorizar componentes grandes o puede introducir regresiones, NO la implementes. Incluyela unicamente en el informe como recomendacion futura." A continuacion se documentan estos hallazgos clasificados por razon de exclusion.

5.1. Sobre-fetching de campos en endpoints de listado

Se identificaron 11 endpoints que retornan todos los campos del modelo cuando el frontend solo usa un subconjunto. Los casos mas claros son `/api/finance` (retorna "contexto" y "description" en cada registro, que son campos de texto libre potencialmente largos), `/api/wellness` (retorna "notes"), `/api/nutrition` (retorna "meals" JSON), y `/api/journal` (retorna "content" y "gratitude"). No se implemento select en estos endpoints porque requeriria verificar cada componente del frontend que consume estos datos para confirmar que ninguno accede a los campos que se excluirian. Un error en esta verificacion causaria campos undefined en el frontend, rompiendo la experiencia de usuario. La recomendacion futura es anadir select progresivamente endpoint por endpoint, validando con pruebas E2E antes de cada despliegue.

5.2. Consultas N+1 en goals/engine.ts

El archivo `src/lib/goals/engine.ts` contiene dos bucles que ejecutan `db.mentorGoal.update()` de forma secuencial: `extractAndPersistGoals` (lineas 507-543) y `updateGoalStates` (lineas 561-614). En ambos casos, cada iteracion del bucle ejecuta una consulta individual de actualizacion. Sin embargo, este archivo tiene 27 errores preexistentes de TypeScript (el modelo "mentorGoal" no existe en el schema Prisma, lo que sugiere que este modulo esta en desarrollo o es codigo muerto). Cualquier modificacion a este archivo corre el riesgo de interactuar mal con estos errores. Se recomienda como trabajo futuro: primero resolver los errores de TypeScript del modulo de goals, luego evaluar si el modelo MentorGoal se implementara o se eliminara, y finalmente batch las actualizaciones si el modulo se mantiene.

5.3. Transacciones faltantes en escrituras multi-paso

Se identificaron 4 ubicaciones con escrituras multi-paso sin transaccion: `/api/onboarding` (5 operaciones secuenciales: `user.update`, `onboardingData.upsert`, `user.update`, `habitLog.findMany`, `habitLog.createMany`, `empireProgress.updateMany`), `/api/stripe/restore` (`user.update` + `subscription.upsert`), `/api/notifications/preferences` (`notificationPreference.upsert` + `pushToken.updateMany` cuando `pushEnabled=false`), y `/api/monthly-closure` (`findUnique` + `conditional update/create`). Sin embargo, la mayoría usan upserts que son atomicos a nivel de fila, y los errores preexistentes en otros modulos sugieren que el proyecto prioriza la estabilidad sobre la correccion atomica. La recomendacion futura es envolver estas operaciones en `db.$transaction()` siguiendo el patron ya establecido en `checkin`, `wellness`, `nutrition`, `journal` y `habits`.

5.4. Error handling: estado actual

La auditoria revelo que el 100% de las rutas API que realizan operaciones de base de datos ya tienen bloques `try/catch` adecuados. Los unicos endpoints sin `try/catch` son `/api/route.ts` y `/api/notifications/sw-config/route.ts`, que son endpoints estaticos que no acceden a la base de datos y por lo tanto no lo necesitan. Este es un indicador de la madurez del proyecto en materia de manejo de errores. No se requieren cambios en esta area.

6. Tabla Resumen de Cambios

#	Archivo	Cambio	Riesgo
1	api/journal/route.ts	take: 100 en GET	Bajo
2	api/wellness/route.ts	Clamp days a [1, 365]	Bajo
3	api/ai/.../messages/route.ts	take: 500 para PREMIUM	Bajo
4	api/ai/threads/route.ts	take: 100 + select en include	Bajo
5	api/habits/route.ts	take: 100 en GET	Bajo
6	api/patterns/route.ts	take: 100 en habitLog	Bajo
7	api/life-memory/route.ts	take: 100 en habitLog	Bajo
8	api/analytics/insights/route.ts	groupBy + raw SQL (5 consultas)	Medio
9	lib/achievements.ts	take: 1095 en allCheckinDates	Bajo
10	prisma/schema.prisma	3 correcciones de sintaxis	Bajo

Tabla 1: Todos los cambios implementados con nivel de riesgo evaluado.

7. Verificacion y Validacion

Todas las optimizaciones fueron verificadas mediante tres mecanismos de validacion. Primero, la validacion del schema con prisma validate confirmo "The schema at prisma/schema.prisma is valid" despues de las correcciones sintacticas. Segundo, la generacion del cliente Prisma con prisma generate completo exitosamente en 519ms, produciendo los tipos actualizados. Tercero, la sincronizacion con la base de datos Neon via prisma db push confirmo "Your database is now in sync with your Prisma schema". Cuarto, la verificacion de tipos con tsc --noEmit confirmo 32 errores de TypeScript, todos preexistentes (goals/engine.ts, timeline/route.ts, NotificationPreferences.tsx, insights.ts, weekly-recap-sender.ts, observability/logger.ts, layout.tsx, prisma.config.ts) y cero errores nuevos introducidos por las optimizaciones. El build de Next.js no pudo completarse debido a la variable de entorno faltante GROQ_API_KEY, que es un problema preexistente independiente de los cambios realizados.

8. Recomendaciones Futuras

Se recomienda priorizar las siguientes mejoras en FASEs posteriores, ordenadas por impacto potencial y complejidad de implementacion. En primer lugar, la adicion progresiva de clausulas select a los endpoints de listado (finance, wellness, nutrition, checkin trends) para reducir el tamaño de respuesta HTTP. Esto requiere una verificacion componente por componente del frontend para garantizar que ningun campo excluido sea accedido. En segundo lugar, la resolucion de los 27 errores de TypeScript preexistentes en goals/engine.ts, que

bloquean cualquier optimización del motor de metas. En tercer lugar, la evaluación de patrones de cursor-based pagination para endpoints con datos potencialmente ilimitados (journal, timeline), reemplazando el simple offset+take por marcadores de posición que mejoran el rendimiento en conjuntos de datos grandes. Finalmente, la consideración de una tabla de agregación diaria para analytics, que eliminaría la necesidad de consultas groupBy sobre la tabla de eventos en tiempo real.